Take-Home Engineering Assessment

Full-Stack Engineer

Time Budget	4–8 hours total. Timebox aggressively. A working MVP beats a polished incomplete.
Preferred Languages	Python, Go, C/C++, React (others acceptable — use what you know)
Submission	Git repository (GitHub / GitLab / Bitbucket) with a README covering setup & run instructions
Evaluation Focus	Correctness → Design → Code clarity → Operability. Aesthetics are not evaluated.

Overview

You will build a small end-to-end system that reads a hardware sensor from your machine, persists the readings, exposes them through a backend API, and renders them in a browser UI. An optional bonus section adds a statistical alarm engine on top.

The emphasis is on a working, well-structured MVP. Each component is intentionally under-specified; the choices you make (and document) are part of what we are assessing.

Part 1 — Read a Hardware Sensor

Goal

Continuously poll any real hardware sensor available on your development machine and emit timestamped readings. The sensor type is your choice — CPU temperature, fan speed, battery state-of-charge, ambient light, accelerometer, etc.

Platform Hints

We are deliberately not prescribing a specific library or API. The list below points you in the right direction per OS; the implementation is yours to decide.

Linux

- The kernel exposes most thermal, power, and fan sensors through the IIO (Industrial I/O) subsystem and the hwmon subsystem under /sys/class.
- libiio is the preferred userspace API for IIO devices. It provides C, Python (pylibiio), and other language bindings.
- The lm-sensors / libsensors stack covers hwmon-exposed sensors (CPU temp, fans, voltages) and is available on virtually every distribution.
- For battery data, the kernel's power_supply class exports attributes such as capacity, voltage_now, and current_now under /sys/class/power_supply/.

macOS

- Apple Silicon and Intel Macs expose thermal and power data through the IOKit framework — specifically through IOServiceGetMatchingService and related IOKit calls.
- SMC (System Management Controller) values — CPU/GPU temperature, fan RPM, battery metrics — are accessible via the SMCKit Swift library or through direct SMC key lookup using IOKit in C.
- For simpler sensors like battery state, the IOPSCopyPowerSourcesInfo / IOPSGetPowerSourceDescription APIs in the IOKit framework provide cross-hardware data without requiring low-level SMC access.
- Open-source tools such as osx-cpu-temp may serve as a useful reference implementation, though we encourage you to call the APIs directly.

Windows

- WMI (Windows Management Instrumentation) exposes temperature, voltage, and fan data through the MSAcpi_ThermalZoneTemperature class and associated providers, queryable via PowerShell, the wmi Python package, or COM automation.
- The OpenHardwareMonitor / LibreHardwareMonitor open-source projects wrap low-level Windows sensor APIs (including WMI, NVAPI, and ACPI). Their source code and WMI server interface are useful reference material.
- For battery data, the Win32_Battery WMI class and the GetSystemPowerStatus Win32 API both provide charge and health information without requiring elevated privileges on most hardware.

> **Note:** You may use any sensor available on your machine. If your machine exposes no convenient hardware sensor, simulating one (e.g. sampling CPU load via OS APIs) is acceptable — document your choice in the README.

Expected Output

- A poller process (daemon, service, script, or thread) that samples the sensor at a configurable interval (default: every 5 seconds).
- Each sample must carry at minimum: a UTC timestamp and the numeric sensor value with its unit.
- The poller must store readings into the persistent store defined in Part 2.

Part 2 — Persistent Store

Goal

Persist sensor readings so they survive process restarts and are queryable by the backend defined in Part 3.

Requirements

- Any storage technology is acceptable: SQLite, PostgreSQL, InfluxDB, Redis, a flat file, etc. Choose based on what you can get running quickly and what fits the query patterns you anticipate.
- The store must be local to the machine (no external cloud dependency required).
- The schema or data model should be documented, even briefly.

> **Note:** SQLite is a natural fit for an MVP — zero infrastructure, works everywhere, and supports the time-range queries needed in Part 4. That said, if you have a preference for something else, use it.

Part 3 — Backend API & Frontend Dashboard

Goal

Expose sensor data through an HTTP API and build a minimal browser UI that lets a user visualise recent readings over time.

Backend

- Any HTTP framework is acceptable (FastAPI, Flask, Gin, Fiber, Express, etc.).
- Provide at minimum one endpoint that returns a time-series of sensor readings for a user-specified time window (e.g. last N minutes).
- Return data in a machine-readable format (JSON is fine).
- Document the endpoints in the README.

Frontend

- Any framework or library is acceptable (React preferred, but plain HTML + JS is fine).
- Display sensor readings over time as a line chart or equivalent time-series visualisation.
- Allow the user to select the time window they want to inspect (e.g. last 5 / 15 / 60 minutes).
- Functionality is the priority. Visual polish, animations, and theming are not evaluated.

> **Note:** The frontend and backend may be the same process or separate. Use whatever approach lets you ship a working system fastest.

Part 4 (Bonus) — Value-Based Alarm Engine

This section is optional. Timebox it to a couple of hours. An MVP-level implementation with documented trade-offs is preferred over an unfinished ambitious design.

Goal

Allow a user to define threshold-based alarms that fire when a statistical aggregate of recent sensor readings crosses a configured limit.

Alarm Specification

Each alarm is defined by three parameters:

Parameter	Allowed Values	Description
Grouping function	avg, max, min, stddev	The statistical aggregate applied to readings within the grouping window.
Grouping duration (X)	Positive integer in minutes	The lookback window over which the aggregate is computed. Example: last 10 minutes.
Alarm threshold (Y)	Numeric	The alarm fires when: aggregate(sensor_value, last X minutes) > Y

Example alarm in pseudo-notation:

```
alarm {
  function  = avg
```

```
  window    = 10    # minutes
  threshold = 75.0  # degrees C
}
# fires when: avg(temp, last 10 min) > 75.0
```

Scope Guidance

The alarm engine is deliberately open-ended. You may scope it up or down based on available time. Some examples across the spectrum:

- Minimal: a polling loop that re-evaluates all alarms every N seconds and logs to stdout when one fires.
- Moderate: alarm definitions stored in the persistent store, evaluated in the backend, and alarm state shown in the UI.
- Extended: CRUD API for alarm management, per-alarm firing history, UI for creation/deletion, optional email or webhook notification.

Document what you built, what you scoped out, and what you would add with more time.

Deliverables & Evaluation

What to Submit

- A git repository containing all source code.
- A README.md with: setup instructions, how to run each component, the sensor chosen and why, schema / data model overview, and any known limitations.
- (Optional) A brief architecture note or diagram if you made non-obvious design decisions worth highlighting.

Evaluation Criteria

Dimension	What We Look At
Correctness	Does the system actually run end-to-end? Are sensor values plausible? Does the UI reflect live data?
Design & Architecture	Are the components cleanly separated? Is the data model reasonable? Are interfaces well-defined?
Code Clarity	Is the code readable and consistently structured? Are error paths handled? Is the README sufficient to run the project cold?
Judgement	Were scope decisions sensible given the time budget? Are trade-offs documented?

Bonus — Alarm Engine	Does the alarm logic correctly implement the specification? How is alarm state managed?

> **Note:** We do not evaluate visual design, UI polish, or aesthetics. A grey-background chart with correct data beats a beautiful UI that shows nothing real.

Questions

If any requirement is ambiguous, state your assumption in the README and proceed. We are more interested in how you navigate ambiguity than in getting every detail exactly right.

Good luck.